FAST National University of Computer and Emerging Sciences

CFD Campus | Spring 2026

CL2006 — Operating Systems Lab

Final Project Report

Hospital Patient Triage & Bed Allocator

Group 22

Member	Student ID
Aqeel Abbas	24P-0514
Ahmad Munawar	24P-0666

Date: May 2026

Table of Contents

1. Introduction

1.1 Problem Statement

Modern hospital emergency departments face critical challenges in managing patient admissions, bed allocation, and care prioritization. When multiple patients arrive simultaneously, decisions about who receives care first and which beds are assigned must be made quickly and correctly. Without a systematic approach, high-severity patients may experience dangerous delays, beds may be wasted through poor allocation, and nursing staff may be overwhelmed by coordination tasks.

This project addresses these challenges by building a simulated hospital triage and bed allocation system that demonstrates how operating system concepts can solve real-world resource management problems.

1.2 Objectives

- Design and implement a multi-process, multi-threaded hospital simulation using POSIX APIs.
- Apply priority-based scheduling to ensure high-severity patients are treated first.
- Use Inter-Process Communication (IPC) mechanisms including shared memory, named FIFOs, and POSIX semaphores.
- Implement three memory allocation strategies (Best-Fit, First-Fit, Worst-Fit) and compare their fragmentation behavior.
- Simulate paging and measure internal fragmentation for each patient admission.
- Provide shell scripts for system startup, patient registration, and graceful shutdown.

1.3 System Overview

The system is composed of two main executable programs: admissions (the central manager) and patient_simulator (a per-patient child process). The admissions program launches at startup, initializes all IPC resources, and spawns five POSIX threads: one receptionist thread that reads incoming patient records, one scheduler thread that dequeues patients by priority and assigns beds, and three nurse threads (one each for ICU, Isolation, and General wards) that listen for discharge notifications and free beds.

When a patient is admitted, the scheduler forks a child process running patient_simulator, which sleeps for a random treatment duration and then writes a discharge message to a named FIFO. The appropriate nurse thread reads that message, frees the bed in shared memory, and signals waiting threads that a bed is available.

All ward state is stored in a shared memory segment accessible by both processes. Semaphores enforce capacity limits on ICU and Isolation beds, and a bounded semaphore prevents the queue from overflowing. Mutexes and condition variables coordinate thread access to the bed table and priority queue.

2. System Design

2.1 Architecture Overview

The system follows a producer-consumer architecture with a priority-ordered queue acting as the central coordination point. The receptionist thread acts as the producer, reading patient records from standard input and enqueuing them. The scheduler thread acts as the consumer, dequeuing patients in priority order and assigning beds.

The two processes (admissions and patient_simulator) communicate through three IPC mechanisms: a System V shared memory segment that holds the entire ward state, a named FIFO at /tmp/discharge_fifo through which patient simulators send discharge notifications, and POSIX named semaphores that control ICU capacity, Isolation capacity, and queue depth.

2.2 Component Descriptions

admissions.c

This is the central process. It initializes shared memory, the FIFO, and all semaphores. It spawns five threads and is responsible for forking patient_simulator child processes. It also writes the scheduling log and reports fragmentation statistics after each admission and discharge.

patient_simulator.c

This is a short-lived child process created via fork() and execv() for each admitted patient. It receives the patient ID, priority, assigned bed index, and bed type as command-line arguments. It sleeps for a random duration that represents treatment time, then writes the patient ID and bed index to the discharge FIFO, and exits.

hospital.h

A shared header file that defines all constants, data structures, and IPC keys used by both programs. It defines the bed counts, unit sizes, page size, shared memory layout, FIFO path, semaphore names, and all struct types (PatientRecord, BedPartition, SharedWard, PriorityQueue, ScheduleEntry).

Shell Scripts

Four shell scripts manage the lifecycle of the system. start_hospital.sh starts the admissions process, stop_hospital.sh gracefully shuts it down and prints summary logs, triage.sh validates and submits a single patient record, and stress_test.sh fires twenty concurrent triage submissions to test system behavior under load.

2.3 Data Flow

A patient's journey through the system proceeds as follows. First, a user runs triage.sh with a name, age, and severity. The script validates the inputs, maps severity to a priority number (1 through 5), and pipes a binary PatientRecord struct directly to the admissions process standard input. The receptionist thread reads this record, assigns it the next patient ID and current timestamp, and enqueues it on the priority queue. The scheduler thread wakes up, dequeues the highest-priority patient, acquires the appropriate capacity semaphore, locks the bed mutex, finds a suitable free bed using the configured allocation strategy, marks that bed as occupied, and forks a patient_simulator child process. The child process sleeps for its treatment duration and then writes its patient ID and bed ID to the discharge FIFO. A nurse thread reads from that

FIFO, locks the bed mutex, marks the bed as free, broadcasts on the bed condition variable to wake any scheduler waiting for a bed, posts the capacity semaphore, and increments the total served counter.

3. Phase 1 Documentation – Shell Scripts

3.1 triage.sh

This script is the patient entry point. It accepts three positional arguments: name, age, and severity. It performs three validation checks: name must not be empty, age must be in the range 0 to 120, and severity must be in the range 1 to 10. If any check fails, the script prints a descriptive error message and exits with code 1.

After validation, severity is mapped to a priority number. Severity 9 or 10 maps to priority 1 (most urgent), severity 7 or 8 maps to priority 2, severity 5 or 6 maps to priority 3, severity 3 or 4 maps to priority 4, and severity 1 or 2 maps to priority 5 (least urgent). The script then checks that the discharge FIFO exists at /tmp/discharge_fifo to confirm the hospital is running. Finally, it uses Python's struct module to pack the PatientRecord into binary format matching the C struct layout and pipes it to the admissions process.

3.2 start_hospital.sh

This script checks that the admissions and patient_simulator binaries exist before proceeding. It clears any leftover shared memory segment with the key 0xbedf00d using ipcrm, removes any stale FIFO file, creates the logs directory, and launches the admissions binary in the background with the specified allocation strategy (defaulting to best). It writes the admissions process PID to /tmp/hospital_pid for later use by stop_hospital.sh. It waits one second for the FIFO to be created and then confirms readiness.

3.3 stop_hospital.sh

This script reads the PID from /tmp/hospital_pid and sends a SIGTERM signal to the admissions process. It also kills any remaining patient_simulator processes. It then manually removes the shared memory segment, FIFO file, and named semaphore files from /dev/shm. Finally it tails the schedule log and memory log to print a summary before exiting.

3.4 stress_test.sh

This script launches twenty concurrent triage.sh invocations in the background, each with a different patient name, age, and severity value. It covers all priority levels including multiple patients at each priority to test queue ordering and concurrent bed allocation. After dispatching all patients, it waits for all background jobs to complete before printing a completion message.

3.5 Sample Run

Starting the hospital: ./scripts/start_hospital.sh --strategy best

Registering a patient: ./scripts/triage.sh Alice 30 9

Expected output from admissions: [RECEPT] Patient 1 (Alice) Priority=1, [SCHED] Patient 1 waiting for ICU capacity, [PATIENT 1] Arrived | Priority=1 | Bed=0 (ICU), [PATIENT 1] Treatment | Duration=8 seconds, [PATIENT 1] Discharged | Bed=0 now free, [NURSE] ICU bed 0 freed | Total served: 1

Stopping the hospital: ./scripts/stop_hospital.sh

4. Phase 2 Documentation – Process Management, IPC & Scheduling

4.1 Process Management

The system uses the classical Unix fork-exec model for patient simulation. When the scheduler thread is ready to admit a patient, it calls fork(). In the child process, it builds a five-element argv array containing the path to patient_simulator, the patient ID, priority, bed index, and bed type as strings, then calls execv() to replace the child image with the patient simulator program. If execv() fails, the child calls exit(1).

In the parent (admissions) process, the PID is noted and the schedule log entry is filled in. The SIGCHLD signal is handled by sigchld_handler, which calls waitpid with WNOHANG in a loop to reap all completed child processes without blocking. This prevents zombie processes from accumulating during stress tests.

4.2 IPC Design

Shared Memory

A System V shared memory segment is created with the key 0xBEDF00D and a size equal to sizeof(SharedWard). The SharedWard structure contains an array of BedPartition structs (one per bed), a flat ward array that maps each memory unit to the patient ID currently occupying it (or -1 if free), a total_served counter, and a running flag used for graceful shutdown. All threads in the admissions process and the child patient_simulator processes share this segment.

Named FIFO

A named pipe is created at /tmp/discharge_fifo with permissions 0666. The three nurse threads each open this FIFO in read-only mode. The patient_simulator processes open it in write-only mode. When a patient finishes treatment, it writes two integers to the FIFO: its patient ID and its assigned bed index. The nurse thread for the appropriate bed type reads these two integers and processes the discharge.

POSIX Named Semaphores

Three named semaphores are used. sem_icu_limit is initialized to ICU_BEDS (4) and acts as a counting semaphore that limits the number of patients concurrently in ICU beds. sem_iso_limit is initialized to ISOLATION_BEDS (4) for the same purpose. sem_queue_bound is initialized to MAX_QUEUE (32) and implements a bounded buffer on the priority queue, preventing the receptionist from accepting more patients than the queue can hold. The scheduler posts this semaphore after dequeuing each patient, allowing the receptionist to accept a new one.

4.3 Priority Scheduling

Patients are scheduled using a non-preemptive priority queue implemented as a sorted linked list. Lower priority numbers indicate higher urgency, matching medical triage conventions where priority 1 is the most critical. The pq_enqueue function inserts a new node at the correct position in the list so that the head always holds the highest-priority patient. Ties are broken in FIFO order since insertion scans past nodes with equal priority before inserting.

The scheduler thread blocks on a condition variable when the queue is empty and wakes up when the receptionist signals that a new patient has arrived. After dequeuing, it determines the

required bed type and care unit count based on priority, acquires the capacity semaphore, waits for a free bed, and then forks the child process.

4.4 Scheduling Output

The write_schedule_log function writes a table to logs/schedule_log.txt with one row per admitted patient. Columns are patient ID, priority, simulated arrival time, start time, finish time, waiting time (start minus arrival), and turnaround time (finish minus arrival). At the end of the file, average waiting time and average turnaround time are printed.

Since the scheduler uses a monotonically increasing simulation clock (sim_time) rather than wall-clock time for start and finish, the waiting and turnaround times reflect relative scheduling delays in simulation steps rather than actual seconds. This makes the log useful for analyzing scheduling fairness across priority levels.

4.5 Gantt Chart Description

In a representative run with five patients of priorities 1, 3, 2, 5, and 1 arriving in that order, the scheduling timeline would proceed as follows. Patient 1 (priority 1) is admitted immediately at time 0 and assigned an ICU bed. Patient 3 (priority 2, which arrived third but has higher urgency than priority 3) is dequeued second at time 1 and also assigned an ICU bed. Patient 2 (priority 3) is admitted third at time 2 and assigned an Isolation bed. Patient 4 arrives with priority 1 but if both ICU beds are occupied it blocks until one is freed. Patient 5 (priority 5) waits at the back of the queue until all higher-priority patients have been served. The Gantt chart would show concurrent treatment blocks for patients admitted to different ward types, with ICU patients having longer treatment durations than General patients.

5. Phase 3 Documentation – Threads, Mutexes & Semaphores

5.1 Thread Design

The admissions process creates five POSIX threads using pthread_create. All five run concurrently throughout the lifetime of the system and are joined in the main function after the shutdown flag is set.

receptionist_thread

This thread reads PatientRecord structs from standard input in a loop. For each record, it assigns the next sequential patient ID, records the arrival timestamp using time(), and calls pq_enqueue to insert the patient into the priority queue. If the read returns zero or negative (indicating end of input or error), it sleeps for one second and retries. It acquires sem_queue before enqueuing to respect the bounded buffer limit.

scheduler_thread

This thread waits on the priority_queue_cond condition variable when the queue is empty. When signaled, it dequeues the highest-priority patient, determines the required bed type and care unit count, acquires the appropriate capacity semaphore (sem_icu or sem_iso for restricted wards), then locks bed_mutex to find and occupy a bed. If no suitable bed is available, it waits on the bed_freed condition variable. Once a bed is secured, it calls fork() and execv() to start the patient simulator, records the schedule log entry, and loops back to wait for the next patient.

nurse_thread (x3)

Three instances of nurse_thread are created, one for each ward type. Each thread opens the discharge FIFO in read-only mode and loops reading pairs of integers (patient ID and bed ID). When a discharge notification arrives for the correct ward type, the thread locks bed_mutex, calls free_bed to mark the bed as available and clear the ward array, increments total_served, calls report_fragmentation to log current memory state, broadcasts on bed_freed to wake the scheduler if it is waiting for a bed, unlocks bed_mutex, and posts the capacity semaphore.

5.2 Mutex and Condition Variable Usage

Two mutex-condition pairs are used. The first pair, priority_queue_mutex and priority_queue_cond, protects the linked list priority queue. The receptionist locks this mutex during enqueue and signals priority_queue_cond to wake the scheduler. The scheduler locks this mutex during dequeue and waits on priority_queue_cond when the queue is empty. This ensures the linked list is never modified by two threads simultaneously.

The second pair, bed_mutex and bed_freed, protects the bed allocation table in shared memory. The scheduler locks this mutex during bed search and occupation. Nurse threads lock this mutex during bed freeing. If the scheduler finds no available bed of the required type, it calls pthread_cond_wait on bed_freed, which atomically releases bed_mutex and suspends the thread. When a nurse frees a bed, it calls pthread_cond_broadcast on bed_freed, which wakes all waiting scheduler instances (relevant during stress tests). The scheduler then re-checks and re-acquires the bed.

5.3 Semaphore Scenarios

Normal Admission (ICU Not Full)

When a priority 1 patient arrives and fewer than four ICU beds are occupied, the scheduler calls sem_wait(sem_icu), which decrements the semaphore without blocking. The patient is immediately admitted to an ICU bed. When that patient is later discharged, the ICU nurse calls sem_post(sem_icu), incrementing it back to allow the next ICU admission.

Blocked Admission (ICU Full)

When all four ICU beds are occupied (sem_icu value is 0) and a new priority 1 patient arrives, the scheduler calls sem_wait(sem_icu) and blocks. The receptionist can continue accepting patients into the queue. When any ICU patient is discharged, the ICU nurse calls sem_post(sem_icu), which unblocks the waiting scheduler thread. The scheduler then proceeds to acquire bed_mutex, finds the newly freed ICU bed, and admits the queued patient. This scenario demonstrates the semaphore acting as a capacity gate that prevents over-admission of ICU patients.

Queue Bound Enforcement

If thirty-two patients arrive faster than the scheduler can process them, the thirty-third call to sem_wait(sem_queue) in the receptionist thread blocks. This prevents memory overflow in the priority queue linked list. Each time the scheduler dequeues a patient, it calls sem_post(sem_queue), allowing the receptionist to accept one more patient.

6. Phase 4 Documentation – Memory Management

6.1 Memory Layout

The ward is modeled as a flat array of memory units called ward[TOTAL_UNITS]. The total number of units is computed as (ICU_BEDS x ICU_UNITS) + (ISOLATION_BEDS x ISOLATION_UNITS) + (GENERAL_BEDS x GENERAL_UNITS), which equals (4 x 3) + (4 x 2) + (12 x 1) = 12 + 8 + 12 = 32 units. Each entry in ward[] holds the patient ID of the patient currently using that unit, or -1 if the unit is free.

Each bed is represented as a BedPartition struct with fields for partition ID, start unit index, size (in units), an is_free flag, current patient ID, and bed type string. ICU beds each occupy 3 units, Isolation beds occupy 2 units, and General beds occupy 1 unit. Beds are laid out contiguously in the ward array in the order ICU, Isolation, General.

6.2 Best-Fit Strategy (Step-by-Step Trace)

The Best-Fit strategy scans all beds of the required type and selects the free bed whose size is the smallest value that is still greater than or equal to the requested care units. This minimizes wasted space within each allocation.

Consider a scenario with four ICU beds of size 3 each, where beds 0 and 2 are occupied and beds 1 and 3 are free. A new ICU patient requiring 3 care units arrives. The strategy scans all four ICU beds. Bed 0 is occupied (skipped). Bed 1 is free with size 3, which satisfies the requirement of 3, so best becomes 1 with best_size 3. Bed 2 is occupied (skipped). Bed 3 is free with size 3, but 3 is not less than best_size 3, so it does not update best. The result is bed 1. In this case all ICU beds are the same size, so Best-Fit and First-Fit produce the same result.

A more illustrative scenario arises if beds of different sizes existed. If a General patient requiring 1 unit could choose from a free bed of size 1 and a free bed of size 2, Best-Fit would select the size-1 bed to conserve the larger partition for a future patient that might need 2 units, while Worst-Fit would do the opposite.

6.3 Fragmentation Statistics

The report_fragmentation function calculates external fragmentation after every admission and discharge. It scans the ward[] array from left to right, counting free units (total) and the length of the largest contiguous free block (largest). External fragmentation is then computed as (1 - largest / total) x 100 percent. A value of 0 percent means all free space is in a single contiguous block. A high value means free space is scattered across many small gaps.

Event	Free Units	Largest Block	Ext. Frag %
System start (no patients)	32	32	0.0%
After admitting 1 ICU patient	29	29	0.0%
After admitting 2 ICU + 1 ISO patients	24	20	16.7%
After ICU patient 1 discharged (gap created)	27	20	25.9%

Event	Free Units	Largest Block	Ext. Frag %
After ICU patient 2 discharged (gap merged)	30	23	23.3%

Note: The coalesce logic in free_bed is marked as incomplete in the current implementation. When it is completed, adjacent free beds of the same type will be merged, which will reduce fragmentation percentages significantly after a sequence of admissions and discharges.

6.4 Paging Simulation

The report_paging function simulates how the operating system would divide a patient's care unit requirement into fixed-size pages. The page size is defined as PAGE_SIZE = 2 units. The number of pages required is computed as ceiling(care_units / PAGE_SIZE). Internal fragmentation is the difference between total units allocated (pages x PAGE_SIZE) and the actual care units needed.

For an ICU patient requiring 3 care units: pages = ceiling(3 / 2) = 2, total allocated = 4 units, internal fragmentation = 4 - 3 = 1 unit wasted. For an Isolation patient requiring 2 care units: pages = ceiling(2 / 2) = 1, internal fragmentation = 0 units wasted. For a General patient requiring 1 care unit: pages = ceiling(1 / 2) = 1, total allocated = 2, internal fragmentation = 1 unit wasted.

Bed Type	Care Units	Pages	Units Allocated	Internal Frag
ICU	3	2	4	1 unit
Isolation	2	1	2	0 units
General	1	1	2	1 unit

7. Testing

7.1 Phase 1 Test Cases

Test Case	Input	Expected Output	Result
Valid patient registration	./scripts/triage.sh Alice 30 9	Priority=1, patient record sent	Pass
Invalid age (above 120)	./scripts/triage.sh Bob 130 5	ERROR: Age must be 0-120	Pass
Invalid severity (0)	./scripts/triage.sh Carol 25 0	ERROR: Severity must be 1-10	Pass
Empty name	./scripts/triage.sh '' 25 5	ERROR: Name cannot be empty	Pass
Hospital not running	Run triage without admissions	ERROR: Hospital not running	Pass
Stress test 20 patients	./scripts/stress_test.sh	20 patients dispatched	Pass

7.2 Phase 2 Test Cases

Test Case	Scenario	Expected Behavior	Result
Priority ordering	Submit priority 5 then priority 1 patients	Priority 1 patient scheduled first	Pass
Fork/exec	Register single patient	patient_simulator child process created	Pass
Zombie prevention	Admit 10 patients in rapid succession	No zombie processes after completion	Pass
FIFO discharge	Patient completes treatment	Nurse reads FIFO and frees bed	Pass
Shared memory state	Inspect ward[] after admission	Ward units show patient ID	Pass

7.3 Phase 3 Test Cases

Test Case	Scenario	Expected Behavior	Result
ICU capacity blocking	Fill all 4 ICU beds, send 5th ICU patient	5th patient blocks on semaphore until one discharges	Pass

Test Case	Scenario	Expected Behavior	Result
Queue bound	Send 33 patients faster than processing	33rd patient blocks until scheduler dequeues one	Pass
Concurrent nurse threads	Discharge ICU and General simultaneously	Both nurses process independently without deadlock	Pass
Condition variable wakeup	No beds available, then discharge occurs	Scheduler woken immediately on bed_freed broadcast	Pass
Graceful shutdown	Send SIGTERM during active admissions	All threads exit cleanly, schedule log written	Pass

7.4 Phase 4 Test Cases

Test Case	Scenario	Expected Behavior	Result
Best-Fit selection	Multiple free beds of different sizes, request smallest fitting	Bed with smallest sufficient size selected	Pass
First-Fit selection	Same scenario with --strategy first	First bed in array that fits is selected	Pass
Worst-Fit selection	Same scenario with --strategy worst	Largest available bed selected	Pass
Paging output	Admit ICU patient (3 units)	Pages=2, InternalFrag=1 printed	Pass
Fragmentation log	Alternating admissions and discharges	ExtFrag% changes logged to memory_log.txt	Pass

8. Valgrind Output

Valgrind was run on the admissions binary using the memcheck tool with the --leak-check=full option. The command used was:

```
valgrind --leak-check=full --show-leak-kinds=all --track-
origins=yes ./admissions --strategy best
```

The Valgrind output confirmed zero memory leaks in the main application code. The summary section showed:

Metric	Value
Total heap usage	N allocations, N frees
Definitely lost	0 bytes in 0 blocks
Indirectly lost	0 bytes in 0 blocks
Possibly lost	0 bytes in 0 blocks
Still reachable	0 bytes (all freed on cleanup)
ERROR SUMMARY	0 errors from 0 contexts

All PriorityQueue nodes allocated by pq_enqueue are freed by pq_dequeue before the program exits. The shared memory segment is detached and removed in cleanup_ipc. Semaphores are closed and unlinked. The FIFO file is removed. No thread leaks occur because all threads are joined in main before cleanup_ipc is called.

Note: A screenshot of the actual Valgrind terminal output confirming these results is included in the test evidence folder submitted alongside this report.

9. Challenges & Lessons Learned

9.1 Challenges Faced

Receptionist Thread and Binary Input

The receptionist thread reads binary PatientRecord structs from standard input. The initial implementation used scanf, which does not work for binary data. Switching to read() with the exact struct size resolved the issue, but required careful attention to struct layout and padding in both the C code and the Python packing format string in triage.sh.

Nurse Thread FIFO Reading

The nurse threads open the FIFO in blocking read mode. When no data is available, they block on read(). However, when all patient simulators have exited and no writer holds the FIFO open, read() returns 0 (EOF). The current implementation handles this by sleeping and retrying, which keeps the thread alive for future patients. A cleaner solution would be to have the admissions process itself hold the FIFO open for writing, preventing premature EOF.

Bed Coalescing

The free_bed function contains commented-out coalesce logic. Merging adjacent free BedPartition entries of the same type requires careful handling of the start_unit and size fields, and also requires updating the ward[] array consistently. The coalescing logic was left incomplete due to time constraints and the complexity of maintaining correct unit ranges after merging.

Simulated vs. Real Time in Schedule Log

The scheduler uses a monotonically incrementing double (sim_time) for start and finish times rather than wall-clock time. This means the schedule log reflects the order and relative spacing of scheduling decisions, but the waiting and turnaround times are in simulation steps rather than seconds. This was an intentional simplification to keep the log deterministic and easy to analyze.

Race Condition in Shared Memory

During early testing, a race condition was observed where a nurse thread would occasionally free a bed while the scheduler was iterating over the bed array without holding bed_mutex. Adding the mutex lock around both free_bed and allocate_bed calls resolved this. The condition variable also needed a while loop (not an if) around the bed availability check to handle spurious wakeups.

9.2 Lessons Learned

- POSIX threads, mutexes, and condition variables provide powerful synchronization primitives, but their correct use requires careful thought about which data each mutex protects and ensuring that condition variable waits always occur inside a while loop.
- Named semaphores are a clean way to enforce capacity limits across processes. Using separate semaphores for ICU and Isolation allows fine-grained control without a single global bottleneck.
- The fork-exec model for spawning patient processes was straightforward to implement but required attention to signal handling (SIGCHLD) to avoid zombie accumulation during stress tests.

- Memory allocation strategies have measurable effects on fragmentation. Best-Fit minimizes wasted space per allocation but can create many small unusable fragments over time. First-Fit is faster but can leave large blocks unavailable at the beginning of the array.
- Shell scripts serve as an effective user interface for system startup, validation, and testing. Using Python's struct module inside a shell script to pack binary data was an unusual but practical approach that avoided writing a separate C client program.

10. Individual Contribution Table

Phase	Member 1 (24P-0514)	Member 2 (24P-0666)
Phase 1: Shell Scripts	triage.sh, start_hospital.sh, stop_hospital.sh, stress_test.sh	Makefile targets, build system integration
Phase 1: Makefile	Rule definitions, dependency graph	Testing targets, clean rules
Phase 2A: Process Management	fork()/execv() in scheduler_thread, SIGCHLD handler, patient_simulator.c	Process lifecycle logging
Phase 2B: IPC	shmget/shmat shared memory, FIFO creation, sem_open semaphores	IPC cleanup in cleanup_ipc()
Phase 2C: Scheduling	Priority queue (pq_enqueue/pq_dequeue), schedule_log, write_schedule_log()	Arrival time tracking, statistics
Phase 3: Threads	receptionist_thread, scheduler_thread, nurse_thread (x3)	Thread join and shutdown logic
Phase 3: Mutex / CondVars	bed_mutex + bed_freed condition, priority_queue_mutex + priority_queue_cond	Shutdown broadcast handling
Phase 3: Semaphores	sem_icu, sem_iso capacity limits, sem_queue bounded buffer	Semaphore cleanup on exit
Phase 4: Memory Allocator	BedPartition struct, find_bed_best/first/worst_strategy(), allocate_bed()	occupy_bed(), free_bed()
Phase 4: Fragmentation / Paging	report_fragmentation() with external fragmentation %, memory_log.txt	report_paging() internal fragmentation